

Virtual Memory

Definition

Virtual Memory is a memory management technique that allows the execution of processes that are larger than the available physical memory (RAM).

It gives each process the illusion of having a large, continuous memory space by using secondary storage (hard disk or SSD) as an extension of RAM.

Instead of loading an entire program into RAM, only the required portions (pages) are loaded. The remaining pages stay on the disk until they are needed.

Why is Virtual Memory Needed?

Physical memory (RAM) is limited. Modern applications such as web browsers, games, IDEs, and databases often require much more memory than the available RAM.

Without virtual memory:

- Large programs cannot execute if they exceed RAM size.
- Fewer programs can run simultaneously.
- Memory utilization becomes inefficient.

Virtual memory overcomes these limitations by allowing the operating system to temporarily store inactive pages on disk.

How Virtual Memory Works

Suppose:

- RAM = 4 GB
- Program size = 8 GB

Instead of loading all 8 GB into RAM:

- Frequently used pages are loaded into RAM.

- Remaining pages remain on the disk.
- When a required page is not found in RAM, it is brought from the disk.

Thus, the operating system continuously swaps pages between RAM and secondary storage.

Virtual memory can be implemented by **demand paging or demand segmentation**.

Out of these two ways, **demand paging** is commonly used as it is easier to implement.

DEMAND PAGING

The **easiest way to implement virtual memory is through demand paging**, where a page is loaded into the memory only when it is needed during program execution.

Pages that are never accessed are never loaded into the memory.

A demand paging system **combines the features of paging with swapping**. Thus, it requires the same **hardware as required** in paging and swapping. That is, it **needs a secondary storage and page table**.

- **Secondary storage:** To facilitate swapping, the entire virtual address space of a process is stored contiguously on a secondary storage device (usually, a disk).

Whenever a process is to be executed, an area on secondary storage device is **allocated to it** on which its pages are copied.

The area is known as **swap space of the process**.

During the execution of a process, whenever a page is required, it is loaded into the main memory from the swap space.

Similarly, when a **process is to be removed** from main memory, it **is written back into the swap space** if it has been modified.

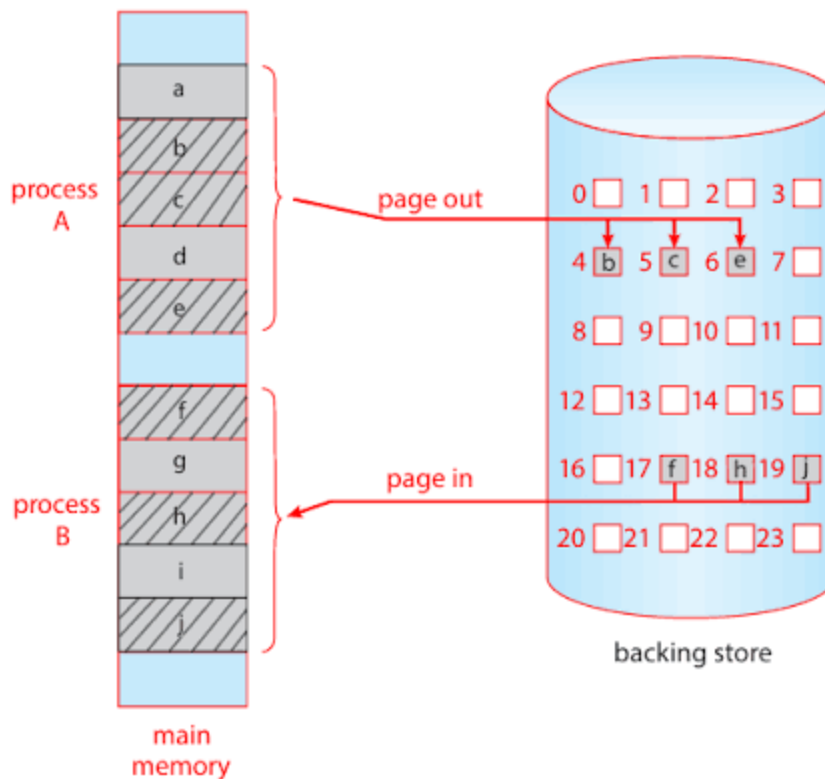


Figure 9.20 Swapping with paging.

- **Page table:** To differentiate the pages that are in memory from those on the disk, and an additional bit **valid** is maintained in each page table entry to indicate whether the page is in the memory or not. If a page is valid (that is, it exists in the virtual address space of the process) and is in the memory, the associated valid bit is set to 1, otherwise it is set to 0. Figure 8.2 shows the page table in demand paging system.

Note: In demand paging system, the term **lazy swapping** is used instead of swapping as the page is never swapped into the memory until it is needed.

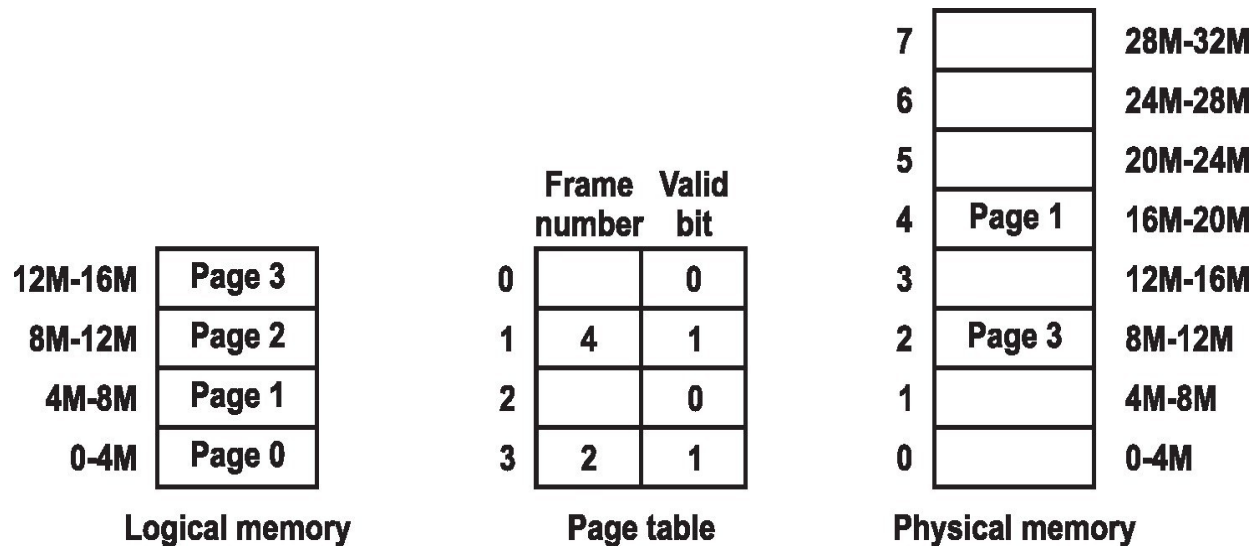


Fig. 8.2 Page Table in Demand Paging System

Whenever a process requests for a page, the virtual address is sent to the MMU.

The MMU checks the valid bit in the page table entry of that page.

If the valid bit is 1 (that is, the requested page is in the memory), it is accessed as in paging.

Otherwise, the MMU raises an interrupt called **page fault** or a **missing page interrupt** and the control is passed to the page fault routine in the operating system.

To handle the page fault, the page fault routine first of all checks if the virtual address for the desired page is valid from its PCB stored in the process table. If it is invalid, it terminates the process giving error. Otherwise, it takes the following steps.

1. Locates for a free page frame in the memory and allocates it to the process.
2. Swaps the desired page into this allocated page frame.
3. Updates the process table and page table to indicate that the page is in the memory.

After performing these steps, the CPU restarts from the instruction that it left off due to the page fault.

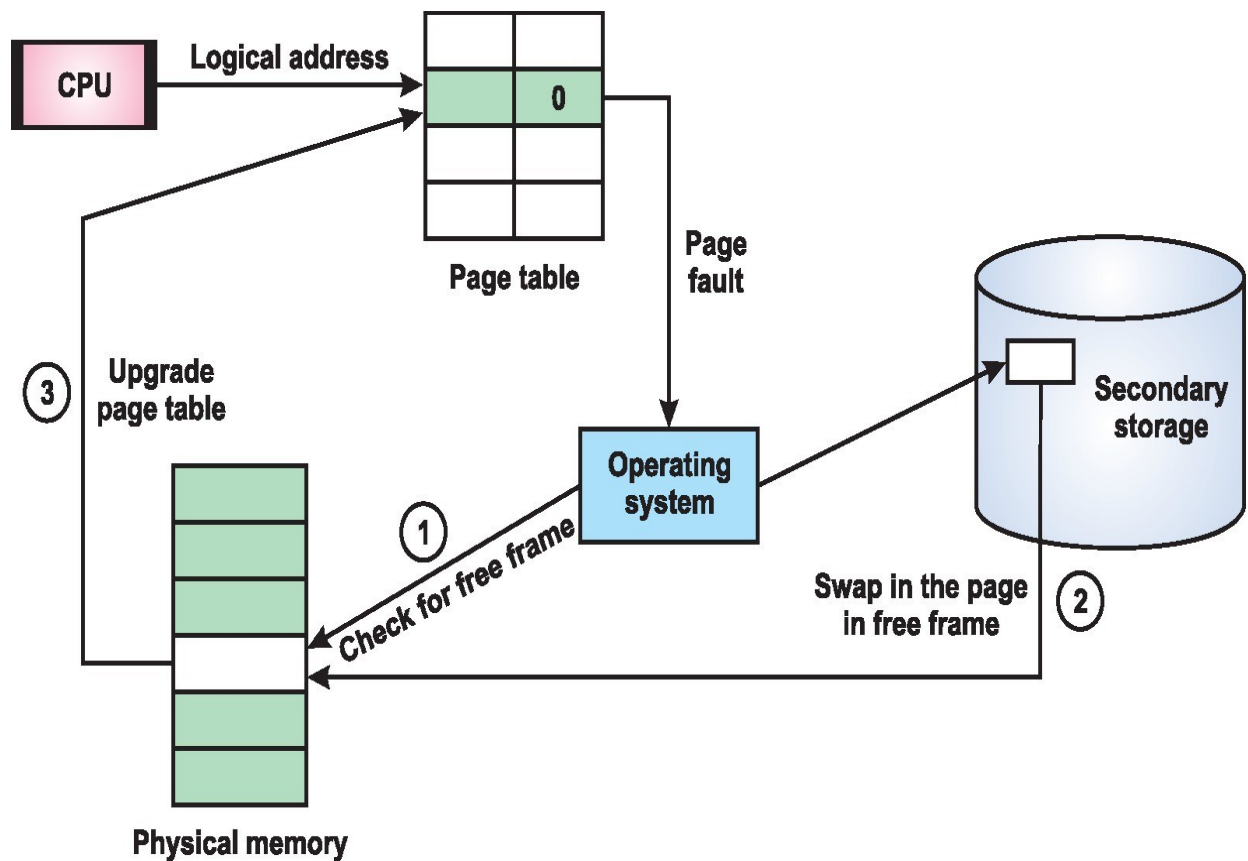


Fig. 8.3 Handling a Page Fault

Note: In demand paging system, the process of loading a page in the memory is known as *page-in* operation instead of *swap-in*. It is because the whole process is not loaded; only some pages are loaded into the memory.

In an **extreme case**, the operating system could decide to start the execution of a process with **no pages in the memory**.

As soon as the instruction pointer is set to the first instruction of the process, the MMU immediately raises a page fault interrupt.

Now, the missing page (page containing the first instruction to be executed) is loaded into the memory. This technique is known as **pure demand paging**, which

is based on the idea: **‘never bring a page until it is required’**.

In some cases, programs access multiple pages from the memory with each instruction. This results in multiple page faults in an instruction. To avoid this and to attain a reasonable performance from demand paging, programs should have locality of reference.

Advantages

- It reduces the swap time since only the required pages are swapped in instead of swapping the whole process.
- It increases the degree of multiprogramming by reducing the amount of physical memory required for a process.
- It minimizes the initial disk overhead as initially not all pages are to be read.
- It does not need extra hardware support.

When a page fault occurs, page fault routine locates a free page frame in the memory and allocates it to the process. However, there is a possibility that the memory is full, that is, no free frame is available for allocation. In that case, the operating system has to evict an existing page from the memory to make space for the desired page to be swapped in. The page to be evicted will be written to the disk depending on whether it has been modified or not. If the page has been modified while in the memory, it is rewritten to the disk; otherwise no rewrite is needed.

To keep track whether the page has been modified, a **modified (M)** bit (also known as **dirty** bit) is added to each page table entry. This bit indicates whether the page has been modified. When a page is first loaded into the memory, this bit is cleared. It is set by the hardware when any word or byte in the page is written into. At the time of page replacement, if dirty bit for a selected page is cleared, it implies that the page has not been modified since it was loaded into the memory. The page frame is written back to the swap space only if dirty bit is set.

The system can select a page frame at random and replace it by the new page. However, if the replaced page is accessed frequently, then another page fault would occur when the replaced page is accessed again resulting in degradation of system performance. Thus, there must be some policy to select a page to be evicted. For this, there are various page replacement algorithms, which can be evaluated by determining the number of page faults using a reference string.

A **reference string** is an ordered list of memory references made by a process. It can be generated by a random-number generator or recording the actual memory references made by an executing program.

To illustrate the page replacement algorithms, consider the reference string as shown in Figure 8.6 for the memory with three page frames. For simplicity, instead of actual memory references, we have considered only the page numbers.

5	0	5	3	5	2	5	0	1	0	7	3
---	---	---	---	---	---	---	---	---	---	---	---

Fig. 8.6 Reference String

8.1.5 FIFO Page Replacement

The first-in, first-out (FIFO) is the simplest page replacement algorithm.

As the name suggests, the first page loaded into the memory is the first page to be replaced. That is, the page is replaced in the order in which it is loaded into the memory.

To illustrate the FIFO replacement algorithm, consider our example reference string shown in Figure 8.6.

5	0	5	3	5	2	5	0	1	0	7	3	Reference string
5	0	–	3	–	2	5	0	1	–	7	3	Swap-in page
5	5	5	5	5	2	2	2	1	1	1	1	} Page frames
–	0	0	0	0	0	5	5	5	5	7	7	
–	–	–	3	3	3	3	0	0	0	0	3	
–	–	–	–	–	5	0	3	2	–	5	0	Swap-out page

Fig. 8.7 FIFO Replacement Algorithm

Assuming that initially all the three frames are empty, the first two references made to page 5 and 0 cause page faults. As a result, they are swapped in the memory. The third reference made to page 5 does not cause page fault as it is already in the memory. The next reference made to page 3 causes a page fault and that page is brought in the memory. The reference to page 2 causes a page fault which results in the replacement of page 5 as it is the oldest page. Now, the oldest page is 0, so reference made to page 5 will replace page 0. This process continues until all the pages of reference string are accessed. It is clear from Figure 8.7 that **there are nine page faults**.

To implement this algorithm, **each page table entry includes the time (called swap-in time) when the page was swapped in the memory**. **When a page is to be replaced, the page with the earliest swap-in time is replaced**. Alternatively, a **FIFO queue** can be created to keep track of all the pages in the memory with the earliest one at the front and the recent at the rear of the queue. **At the time of page fault, the page at the front of the queue is removed and the newly arrived page is added to the rear of the queue**.

The FIFO page replacement algorithm **is easier to implement** as compared to

all other replacement algorithms. However, it is rarely used as it is not very efficient. Since it does not consider the pattern of the usage of a page, a frequently used one may be replaced resulting in more page faults.

Belady's Anomaly

FIFO page replacement algorithm suffers from **Belady's anomaly**—a situation in which increasing the number of page frames results in more page faults. To illustrate this, consider the reference string containing five pages, numbered 2 to 6 (see Figure 8.8). From this figure, it is clear that with three page frames, a total of nine page faults occur. On the other hand, with four page frames, a total of ten page faults occur.

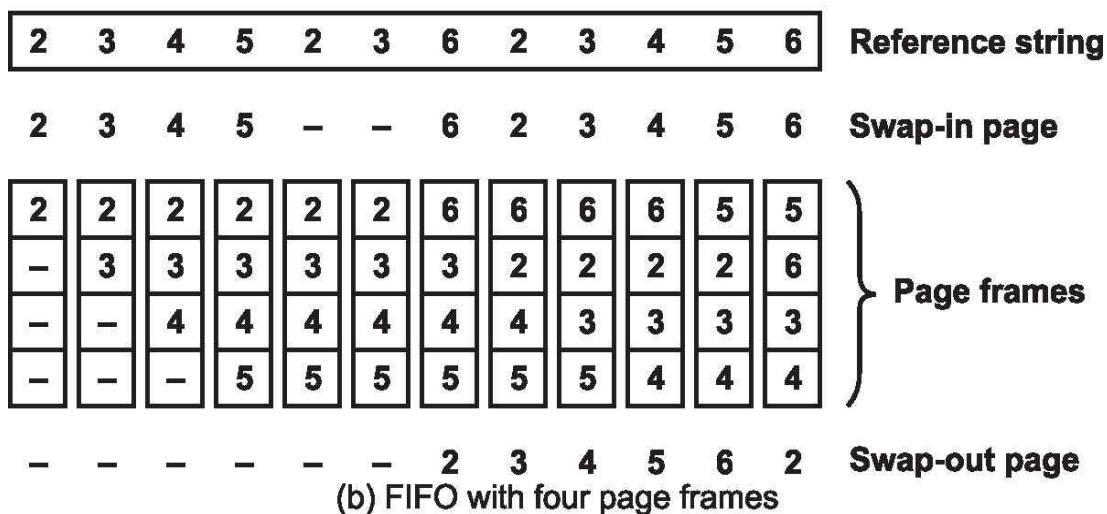
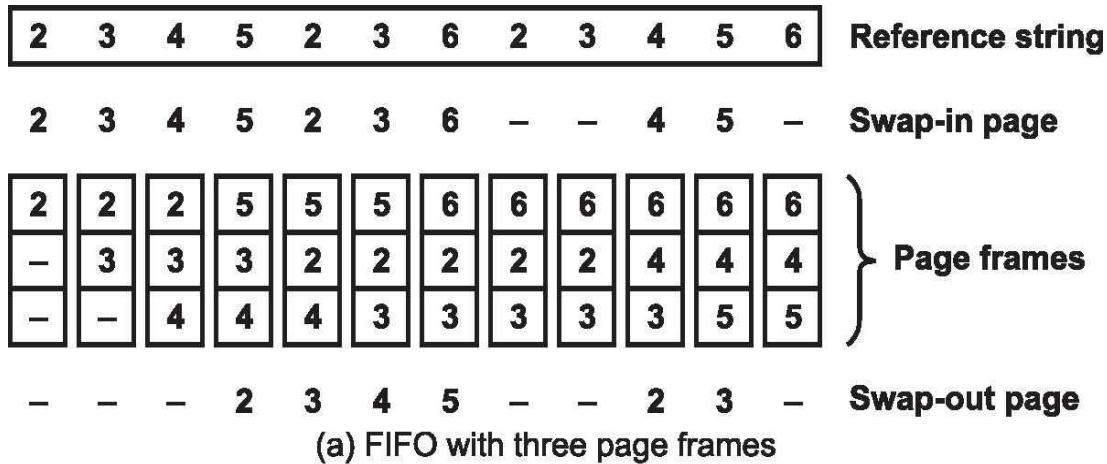


Fig. 8.8 Belady's Anomaly

8.5.1 Optimal Page Replacement

Optimal page replacement (OPT) algorithm is the **best possible page replacement algorithm**.

The basic idea behind this algorithm is that **whenever a page fault occurs, some pages are in memory; out of these pages, one will be referenced at the next instruction while other pages may not be referenced until the execution of certain number of instructions**.

In case of page replacement, **the page that is referenced at last will be replaced**. That is, the **page to be referenced in the most distant future is replaced**. For this, each page can be labeled in the memory with the number of instructions to be executed before that page is referenced for the first time. The page with the highest label is replaced from the memory.

To illustrate this algorithm, consider our sample reference string (see [Figure 8.6](#)). Like FIFO, the first two references made to page 5 and 0 cause page faults. As a result, they are swapped into the memory. The third reference made to page 5 does not cause page fault as it is already in the memory. The reference made to page 3 causes a page fault and thus is swapped into the memory. However, the reference made to page 2 replaces page 3 because page 3 is required at the last instruction whereas pages 5 and 0 are required at next instructions. The page faults and the pages swapped in and swapped out for all the page references are shown in [Figure 8.9](#). This algorithm causes seven page faults.

5	0	5	3	5	2	5	0	1	0	7	3	Reference string
5	0	–	3	–	2	–	–	1	–	7	3	Swap-in page
5	5	5	5	5	5	5	5	1	1	1	1	} Page frames
–	0	0	0	0	0	0	0	0	0	7	7	
–	–	–	3	3	2	2	2	2	2	2	3	
–	–	–	–	–	3	–	–	2	–	1	0	Swap-out page

Fig. 8.9 Optimal Page Replacement Algorithm

The **advantage** of this algorithm is that it causes least page faults as compared to other algorithms.

The disadvantage of this algorithm is that its implementation requires prior knowledge of which page will be referenced next. Though this algorithm is not used in systems practically, it is used as the basis for comparing performance of other algorithms.

***Note:** To implement OPT, a program can be executed on a simulator and all the page references are recorded. Using the page reference records obtained during first run, it can be implemented at the second run.*

8.5.2 LRU Page Replacement

The least recently used (LRU) algorithm is an approximation to the optimal algorithm. Unlike optimal algorithm, it uses the recent past behavior of the program to predict the near future.

It is based on the assumption that the page that has been used in the last few instructions will probably be referenced in the next few instructions. Thus, it replaces the page that has not been referenced for the longest time.

Consider the same sample reference string of Figure 8.6. As a result of LRU algorithm, the page faults and the pages swapped in and swapped out for all the page references are shown in Figure 8.10. Up to five references, page faults are the same as that of optimal algorithm. When a reference is made to page 2, page 0 is replaced as it was least recently used. However, after page 5, it is being used again leading to a page fault. Regardless of this, the number of page faults is eight which is less than in case of FIFO.

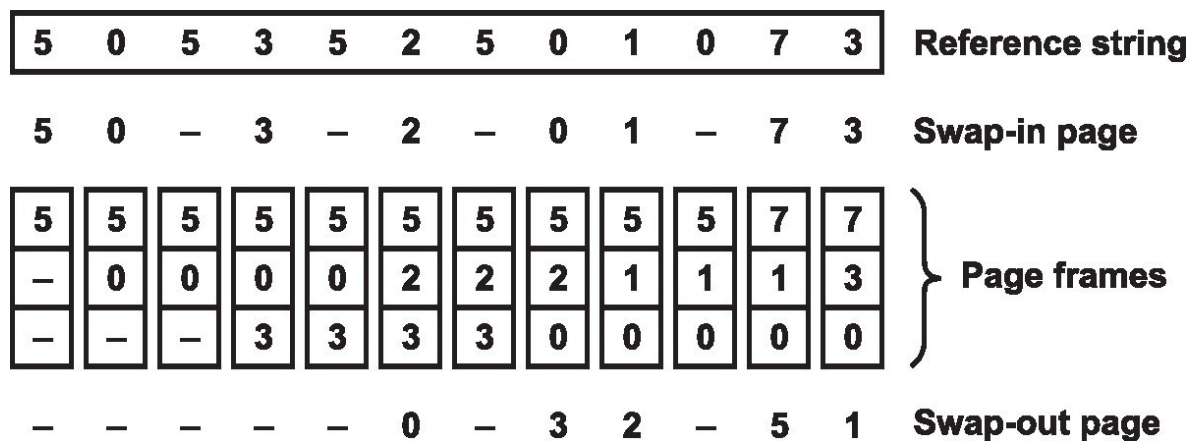


Fig. 8.10 LRU Page Replacement Algorithm

One way to implement LRU is to **maintain a linked list** of all the pages in the memory; with the **most recently used page kept at the head and the least recently used page kept at the tail of the list**. Whenever a page is to be replaced, **it is deleted from the tail** of the linked list and the **new page is inserted at the head** of the linked list. The **problem** with this implementation is that **it requires updating the list at every page reference irrespective of whether the page fault occurs or not**. It is because whenever a page in the memory is referenced, being the most recent page, it is removed from its current position and inserted at the head of the linked list. **This results in extra overhead**.

Alternatively, the **hardware can be equipped with a counter**. This **counter is incremented by one after each instruction**. The page table has a field to store the value of the counter. Whenever a page is referenced, the current value of the counter is copied to that field in the page table entry for that page. Whenever a page is to be replaced, this algorithm searches the page table for the entry having the lowest counter value (means the least recently used page) and replaces that page.

Clearly, it has **less page faults as compared to FIFO algorithm**. Moreover, it **does not suffer from Belady's anomaly**. Thus, it is an improvement on FIFO algorithm and is used in many systems. The **disadvantage** of this algorithm is that **it is time consuming when implemented using linked list**. Otherwise, **it needs extra hardware support for its implementation**.

Note: Both the optimal and LRU algorithms belong to a special class of page replacement algorithms called **stack algorithms** that **never exhibit Belady's anomaly**.

Most Recently Used (MRU)

In this algorithm, page will be replaced which has been used recently. Belady's anomaly can occur in this algorithm.

Consider the page reference string 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4-page frames. Find number of page faults using MRU Page Replacement Algorithm.

Page reference		7,0,1,2,0,3,0,4,2,3,0,3,2,3												No. of Page frame - 4	
7	0	1	2	0	3	0	4	2	3	0	3	2	3		
			2	2	2	2	2	2	3	0	3	2	3		
		1	1	1	1	1	1	1	1	1	1	1	1		
	0	0	0	0	3	0	4	4	4	4	4	4	4		
7	7	7	7	7	7	7	7	7	7	7	7	7	7		
Miss	Miss	Miss	Miss	Hit	Miss	Miss	Miss	Hit	Miss	Miss	Miss	Miss	Miss		
Total Page Fault = 12															

Most Recently Used - Page Replacement

Initially, all slots are empty, so when 7 0 1 2 are allocated to the empty slots
---> 4 Page faults

0 is already there so--> 0 page fault

when 3 comes it will take place of 0 because it is most recently used ---> 1 Page fault

when 0 comes it will take place of 3 ---> 1 Page fault

when 4 comes it will take place of 0 ---> 1 Page fault

2 is already in memory so ---> 0 Page fault

when 3 comes it will take place of 2 ---> 1 Page fault

when 0 comes it will take place of 3 ---> 1 Page fault

when 3 comes it will take place of 0 ---> 1 Page fault

when 2 comes it will take place of 3 ---> 1 Page fault

when 3 comes it will take place of 2 ---> 1 Page fault

Example 1 Consider the following page reference string: 1,

2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6

How many page faults would occur for the following replacement algorithms, assuming one, two, three and four frames?

- (i) LRU replacement
- (ii) FIFO replacement
- (iii) Optimal replacement

Remember that all frames are initially empty, so your first unique pages will all cost one fault each.

Solution: In case of one page frame, each page reference causes a page fault. As a result, there will be 20 page faults for all the three replacement algorithms.

Page frames =2

LRU replacement causes 18 page faults as shown in the figure given below.

1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6	Reference string
1	2	3	4	2	1	5	6	2	1	-	3	7	6	3	2	1	-	3	6	Swap-in page
1	1	3	3	2	2	5	5	2	2	2	2	7	7	3	3	1	1	3	3	} Page frames
-	2	2	4	4	1	1	6	6	1	1	3	3	6	6	2	2	2	2	6	
-	-	1	2	3	4	2	1	5	6	-	1	2	3	7	6	3	-	1	2	Swap-out page

FIFO replacement also causes 18 page faults (see the figure given below).

1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6	Reference string
1	2	3	4	2	1	5	6	2	1	-	3	7	6	3	2	1	-	3	6	Swap-in page
1	1	3	3	2	2	5	5	2	2	2	3	3	6	6	2	2	2	3	3	} Page frames
-	2	2	4	4	1	1	6	6	1	1	1	7	7	3	3	1	1	1	6	
-	-	1	2	3	4	2	1	5	6	-	2	1	3	7	6	3	-	2	1	Swap-out page

Optimal replacement, on the other hand, causes only 15 page faults (see the figure given below).

1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6	Reference string
1	2	3	4	-	1	5	6	-	1	-	3	7	6	-	2	1	-	3	6	Swap-in page
1	1	3	4	4	1	5	6	6	1	1	3	3	3	3	3	1	1	1	6	} Page frames
-	2	2	2	2	2	2	2	2	2	2	2	7	6	6	2	2	2	3	3	
-	-	1	3	-	4	1	5	-	6	-	1	2	7	-	6	3	-	2	1	Swap-out page

Page frames =3

LRU replacement causes 15 page faults as shown in figure given below.

1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6	Reference string
1	2	3	4	-	1	5	6	2	1	-	3	7	6	-	2	1	-	-	6	Swap-in page
1	1	1	4	4	4	5	5	5	1	1	1	7	7	7	2	2	2	2	2	} Page frames
-	2	2	2	2	2	2	6	6	6	6	3	3	3	3	3	3	3	3	3	
-	-	3	3	3	1	1	1	2	2	2	2	2	6	6	6	1	1	1	6	
-	-	-	1	-	3	4	2	1	5	-	6	1	2	-	7	6	-	-	1	Swap-out page

FIFO replacement causes 16 page faults as shown in figure given below.

1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6	Reference string
1	2	3	4	-	1	5	6	2	1	-	3	7	6	-	2	1	-	3	6	Swap-in page
1	1	1	4	4	4	4	6	6	6	6	3	3	3	3	2	2	2	2	6	} Page frames
-	2	2	2	2	1	1	1	2	2	2	2	7	7	7	7	1	1	1	1	
-	-	3	3	3	3	5	5	5	1	1	1	1	6	6	6	6	6	3	3	
-	-	-	1	-	2	3	4	1	5	-	6	2	1	-	3	7	-	6	2	Swap-out page

Optimal replacement causes 11 page faults as shown in figure given below.

1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6	Reference string
1	2	3	4	-	-	5	6	-	-	-	3	7	-	-	2	1	-	-	6	Swap-in page
1	1	1	1	1	1	1	1	1	1	1	3	3	3	3	3	3	3	3	6	} Page frames
-	2	2	2	2	2	2	2	2	2	2	2	7	7	7	2	2	2	2	2	
-	-	3	4	4	4	5	6	6	6	6	6	6	6	6	6	1	1	1	1	
-	-	-	3	-	-	4	5	-	-	-	1	2	-	-	7	6	-	-	3	Swap-out page

Page frames = 4

LRU replacement causes 10 page faults as shown in figure given next.

1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6	Reference string
1	2	3	4	-	-	5	6	-	-	-	3	7	6	-	-	1	-	-	-	Swap-in page
1	1	1	1	1	1	1	1	1	1	1	1	1	6	6	6	6	6	6	6	} Page frames
-	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	
-	-	3	3	3	3	5	5	5	5	5	3	3	3	3	3	3	3	3	3	
-	-	-	4	4	4	4	6	6	6	6	6	7	7	7	7	1	1	1	1	} Page frames
-	-	-	-	-	-	3	4	-	-	-	5	6	1	-	-	7	-	-	-	
-	-	-	-	-	-	3	4	-	-	-	5	6	1	-	-	7	-	-	-	

FIFO replacement causes 14 page faults as shown in figure given below.

1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6
Reference string																			
1	2	3	4	-	-	5	6	2	1	-	3	7	6	-	2	1	-	3	-
Swap-in page																			
1	1	1	1	1	1	5	5	5	5	5	3	3	3	3	3	1	1	1	1
-	2	2	2	2	2	2	6	6	6	6	6	7	7	7	7	7	7	3	3
-	-	3	3	3	3	3	3	2	2	2	2	2	6	6	6	6	6	6	6
-	-	-	4	4	4	4	4	4	1	1	1	1	1	1	2	2	2	2	2
Page frames																			
-	-	-	-	-	-	1	2	3	4	-	5	6	2	-	1	3	-	7	-
Swap-out page																			

Optimal replacement causes 8 page faults as shown in figure given below.

1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6
Reference string																			
1	2	3	4	-	-	5	6	-	-	-	-	7	-	-	-	1	-	-	-
Swap-in page																			
1	1	1	1	1	1	1	1	1	1	1	1	7	7	7	7	1	1	1	1
-	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
-	-	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3
-	-	-	4	4	4	4	5	6	6	6	6	6	6	6	6	6	6	6	6
Page frames																			
-	-	-	-	-	-	4	5	-	-	-	-	1	-	-	-	7	-	-	-
Swap-out page																			

Thrashing

What is Thrashing?

Thrashing is a condition in a virtual memory system in which the operating system spends **more time swapping pages between the main memory (RAM) and secondary storage (disk)** than executing the actual instructions of processes.

It occurs when the **page fault rate becomes very high**, causing the CPU to wait continuously for pages to be loaded from the disk. As a result, system performance drops significantly.

Definition

Thrashing is a condition in which excessive page faults cause the operating system to spend more time transferring pages between RAM and disk than executing processes.

Why Does Thrashing Occur?

Thrashing occurs due to the following reasons:

1. **Insufficient Physical Memory (RAM):**
 - o The available RAM is too small to hold the active pages (working sets) of all running processes.
2. **High Degree of Multiprogramming:**
 - o Too many processes compete for the limited memory, forcing frequent page replacements.
3. **Poor Page Replacement Decisions:**
 - o Frequently used pages may be replaced, causing them to be loaded again shortly afterward.
4. **Large Working Set:**
 - o If a process requires many pages simultaneously and they cannot all fit into RAM, continuous swapping occurs.

Example

Suppose:

- RAM can hold **10 pages**.
- Three processes together require **25 active pages**.

When Process A needs Page 5, the OS removes Page 8.

A few moments later, another process needs Page 8 again.

The OS removes another page and reloads Page 8.
This continuous cycle of removing and loading pages causes thrashing.

How Does the CPU Detect Thrashing?

The CPU itself **does not directly detect thrashing**. Instead, the **Operating System** monitors system performance and identifies thrashing using **several indicators**.

1. High Page Fault Rate

The operating system continuously monitors the number of page faults.

- A few page faults are normal.
- **A sudden increase indicates that processes frequently request pages that are not in memory.**

A consistently high page fault rate is the primary indication of thrashing.

2. Low CPU Utilization

When thrashing occurs:

- The CPU spends most of its time waiting for disk I/O.
- Very little useful computation takes place.

The operating system observes:

- **High disk activity**
- **Low CPU utilization**

This combination strongly indicates thrashing.

3. Excessive Disk Activity

The operating system monitors disk operations.

If the disk is continuously busy transferring pages between RAM and secondary storage, it indicates excessive paging.

4. Continuous Page Replacement

The operating system notices that pages are repeatedly loaded and removed from memory within a short period.

This frequent page replacement is another symptom of thrashing.

5. Monitoring Page Fault Frequency (PFF)

The operating system **measures the page fault rate of each process.**

- **If the page fault frequency exceeds a predefined threshold**, the process may be causing thrashing.
- The OS can then allocate more frames or reduce the number of active processes.

Indicators of Thrashing

Indicator	Observation
Page Fault Rate	Very High
CPU Utilization	Very Low
Disk Activity	Very High
Process Execution	Very Slow
Page Replacement	Very Frequent

Techniques to Prevent Thrashing

Increase Physical Memory (RAM)

The **simplest solution is to install more RAM.**

With more memory:

- More pages can remain in RAM.
- Fewer page faults occur.
- Disk accesses decrease.

Advantage: Most effective solution.

Several techniques can be used to reduce or prevent thrashing.

Reduce the Degree of Multiprogramming

If too many processes are running simultaneously, the operating system temporarily suspends or swaps out some processes.

This frees memory for the remaining processes.

As a result:

- Each process receives more memory.
- Page faults decrease.

Several techniques can be used to reduce or prevent thrashing.

1. Locality

Thrashing can be prevented if each process is allocated as much memory (frames) as it requires.

But how should the operating system know the memory requirement (number of frames required) of a process?

The solution to this problem is influenced by two opposing factors: over-commitment and under-commitment of memory.

If a process is allocated more number of frames (**over-commitment**) than it requires, only few page faults would occur.

The process performance would be good; however, the degree of multiprogramming would be low.

As a result, CPU utilization and system performance would be poor.

In contrast, **under-commitment** of memory to a process causes high page-fault rate which would result in poor process performance.

Thus, for better system performance, it is necessary to allocate appropriate number of frames to each process.

A clue about the number of frames needed by a process can be obtained using the **locality model** of a process execution.

Locality model states that while a process executes, it moves from locality to locality.

Locality is defined as the set of pages that are actively used together.

It is a **dynamic property** in the sense that the identity of the particular pages that form the actively used set varies with time. That is, the program moves from one locality to another during its execution.

***Note:** Localities of a process may coincide partially or wholly.*

The **principle of locality** ensures that not too many page faults would occur if the pages in the current locality of a process are present in the memory.

However, it does not rule out page faults totally.

Once all pages in the current locality of a process are in the memory, page fault would not occur until the process changes locality.

On the other hand, if a process has not been allocated enough frames to accommodate its current locality, thrashing would result.

2. Working Set Model

The **Working Set** of a process is the set of pages that the process is actively using during a particular time interval.

The operating system ensures that each process has enough frames to hold its working set.

If enough frames are not available:

- The process is suspended.
- More frames become available for other processes.

This significantly reduces page faults.

Advantages

- Reduces unnecessary page replacement.
- Improves CPU utilization.
- Prevents thrashing effectively.

3. Page Fault Frequency (PFF) Scheme

The operating system **maintains an acceptable range for page faults**(Threshold).

If Page Fault Rate is High

- Allocate additional page frames.

If Page Fault Rate is Low

- Remove unnecessary frames and allocate them to other processes.
-

Thus, **memory allocation is adjusted dynamically according to each process's needs.**

4. Use Efficient Page Replacement Algorithms

A good page replacement algorithm reduces unnecessary page faults.

Examples include:

- Least Recently Used (LRU)
- Optimal Page Replacement (used as a theoretical benchmark)
- Clock (Second-Chance) Algorithm

These algorithms try to keep frequently used pages in memory.